

We will now use the same interface for more advanced interpolation methods.

CITED REFERENCES AND FURTHER READING:

Knuth, D.E. 1997, *Sorting and Searching*, 3rd ed., vol. 3 of *The Art of Computer Programming* (Reading, MA: Addison-Wesley), §6.2.1.

3.2 Polynomial Interpolation and Extrapolation

Through any two points there is a unique line. Through any three points there is a unique quadratic. *Et cetera*. The interpolating polynomial of degree $M - 1$ through the M points $y_0 = f(x_0), y_1 = f(x_1), \dots, y_{M-1} = f(x_{M-1})$ is given explicitly by Lagrange's classical formula,

$$\begin{aligned} P(x) = & \frac{(x - x_1)(x - x_2)\dots(x - x_{M-1})}{(x_0 - x_1)(x_0 - x_2)\dots(x_0 - x_{M-1})} y_0 \\ & + \frac{(x - x_0)(x - x_2)\dots(x - x_{M-1})}{(x_1 - x_0)(x_1 - x_2)\dots(x_1 - x_{M-1})} y_1 + \dots \\ & + \frac{(x - x_0)(x - x_1)\dots(x - x_{M-2})}{(x_{M-1} - x_0)(x_{M-1} - x_1)\dots(x_{M-1} - x_{M-2})} y_{M-1} \end{aligned} \quad (3.2.1)$$

There are M terms, each a polynomial of degree $M - 1$ and each constructed to be zero at all of the x_i 's except one, at which it is constructed to be y_i .

It is not terribly wrong to implement the Lagrange formula straightforwardly, but it is not terribly right either. The resulting algorithm gives no error estimate, and it is also somewhat awkward to program. A much better algorithm (for constructing the same, unique, interpolating polynomial) is *Neville's algorithm*, closely related to and sometimes confused with *Aitken's algorithm*, the latter now considered obsolete.

Let P_0 be the value at x of the unique polynomial of degree zero (i.e., a constant) passing through the point (x_0, y_0) ; so $P_0 = y_0$. Likewise define $P_1, P_2, \dots, P_{M-1}$. Now let P_{01} be the value at x of the unique polynomial of degree one passing through both (x_0, y_0) and (x_1, y_1) . Likewise $P_{12}, P_{23}, \dots, P_{(M-2)(M-1)}$. Similarly, for higher-order polynomials, up to $P_{012\dots(M-1)}$, which is the value of the unique interpolating polynomial through all M points, i.e., the desired answer. The various P 's form a "tableau" with "ancestors" on the left leading to a single "descendant" at the extreme right. For example, with $M = 4$,

$$\begin{array}{ccccccc} x_0 : & y_0 = P_0 & & & & & \\ & & P_{01} & & & & \\ x_1 : & y_1 = P_1 & & P_{012} & & & \\ & & P_{12} & & P_{0123} & & \\ x_2 : & y_2 = P_2 & & P_{123} & & & \\ & & P_{23} & & & & \\ x_3 : & y_3 = P_3 & & & & & \end{array} \quad (3.2.2)$$

Neville's algorithm is a recursive way of filling in the numbers in the tableau a column at a time, from left to right. It is based on the relationship between a

“daughter” P and its two “parents,”

$$P_{i(i+1)\dots(i+m)} = \frac{(x - x_{i+m})P_{i(i+1)\dots(i+m-1)} + (x_i - x)P_{(i+1)(i+2)\dots(i+m)}}{x_i - x_{i+m}} \quad (3.2.3)$$

This recurrence works because the two parents already agree at points $x_{i+1} \dots x_{i+m-1}$.

An improvement on the recurrence (3.2.3) is to keep track of the small *differences* between parents and daughters, namely to define (for $m = 1, 2, \dots, M-1$),

$$\begin{aligned} C_{m,i} &\equiv P_{i\dots(i+m)} - P_{i\dots(i+m-1)} \\ D_{m,i} &\equiv P_{i\dots(i+m)} - P_{(i+1)\dots(i+m)}. \end{aligned} \quad (3.2.4)$$

Then one can easily derive from (3.2.3) the relations

$$\begin{aligned} D_{m+1,i} &= \frac{(x_{i+m+1} - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \\ C_{m+1,i} &= \frac{(x_i - x)(C_{m,i+1} - D_{m,i})}{x_i - x_{i+m+1}} \end{aligned} \quad (3.2.5)$$

At each level m , the C 's and D 's are the corrections that make the interpolation one order higher. The final answer $P_{0\dots(M-1)}$ is equal to the sum of *any* y_i plus a set of C 's and/or D 's that form a path through the family tree to the rightmost daughter.

Here is the class implementing polynomial interpolation or extrapolation. All of its “support infrastructure” is in the base class `Base_interp`. It needs only to provide a `rawinterp` method that contains Neville's algorithm.

`struct Poly_interp : Base_interp`

`interp_1d.h`

Polynomial interpolation object. Construct with **x** and **y** vectors, and the number M of points to be used locally (polynomial order plus one), then call `interp` for interpolated values.

```
{
    Doub dy;
    Poly_interp(VecDoub_I &xv, VecDoub_I &yv, Int m)
        : Base_interp(xv,&yv[0],m), dy(0.) {}
    Doub rawinterp(Int jl, Doub x);
};
```

`Doub Poly_interp::rawinterp(Int jl, Doub x)`

Given a value x , and using pointers to data xx and yy , this routine returns an interpolated value y , and stores an error estimate dy . The returned value is obtained by mm -point polynomial interpolation on the subrange $xx[jl..jl+mm-1]$.

```
{
    Int i,m,ns=0;
    Doub y,den,dif,dift,ho,hp,w;
    const Doub *xa = &xx[jl], *ya = &yy[jl];
    VecDoub c(mm),d(mm);
    dif=abs(x-xa[0]);
    for (i=0;i<mm;i++) {           Here we find the index ns of the closest table entry,
        if ((dift=abs(x-xa[i])) < dif) {
            ns=i;
            dif=dift;
        }
        c[i]=ya[i];                 and initialize the tableau of c's and d's.
        d[i]=ya[i];
    }
    y=ya[ns--];                    This is the initial approximation to y.
    for (m=1;m<mm;m++) {           For each column of the tableau,
```